

Angular built-in Directives

What Are Directives in Angular?

In Angular, a **directive** is a class that **adds behavior to elements in the DOM**. Directives allow you to:

- Show or hide elements
- Loop through collections
- Dynamically apply styles or classes
- Modify element behavior

Angular provides **built-in directives**, so you do not need to write them from scratch.

Classification of Angular Built-in Directives

Angular built-in directives fall into **three categories**:

1. **Structural Directives**
 2. **Attribute Directives**
 3. **Component Directives** (Components themselves)
-

1. Structural Directives (* syntax)

Structural directives **change the DOM structure** by adding or removing elements.

They usually start with * (syntactic sugar).

Common Structural Directives

Directive	Purpose
*ngIf	Conditionally renders elements
*ngFor	Iterates over collections
*ngSwitch	Switch-case rendering

Directive	Purpose
*ngSwitchCase	Case matching
*ngSwitchDefault	Default case

***ngIf – Conditional Rendering**

```
<p *ngIf="isLoggedIn">Welcome User</p>
```

How it works

- If isLoggedIn === true → element is added to DOM
- If false → element is completely removed

ngIf with else

```
<p *ngIf="isLoggedIn; else guest">
```

```
    Welcome Back
```

```
</p>
```

```
<ng-template #guest>
```

```
    <p>Please Login</p>
```

```
</ng-template>
```

***ngFor – Looping Through Data**

```
<ul>
```

```
    <li *ngFor="let item of products; index as i">
```

```
        {{ i + 1 }} - {{ item.name }}
```

```
    </li>
```

```
</ul>
```

ngFor Useful Variables

- index
- first
- last
- even
- odd

```
<li *ngFor="let user of users; first as isFirst">
  {{ isFirst ? 'First:' : '' }} {{ user.name }}
</li>
```

trackBy (Performance Optimization)

Angular 20+ strongly recommends trackBy for large lists.

```
<li *ngFor="let item of items; trackBy: trackById">
  {{ item.name }}
</li>

trackById(index: number, item: any) {
  return item.id;
}
```

***ngSwitch – Multiple Conditions**

```
<div [ngSwitch]="role">
  <p *ngSwitchCase="'admin'">Admin Panel</p>
  <p *ngSwitchCase="'user'">User Dashboard</p>
  <p *ngSwitchDefault>Guest Access</p>
</div>
```

2. Attribute Directives

Attribute directives **change the appearance or behavior** of an element without changing DOM structure.

Common Attribute Directives

Directive Purpose

ngClass Add/remove CSS classes

ngStyle Apply inline styles

ngModel Two-way binding (Forms)

ngClass – Dynamic CSS Classes

```
<div [ngClass]="{  
  'active': isActive,  
  'disabled': isDisabled  
}">  
  Status  
</div>
```

Array Syntax

```
<div [ngClass]="['box', 'highlight']"></div>
```

ngStyle – Dynamic Inline Styles

```
<p [ngStyle]="{  
  color: isError ? 'red' : 'green',  
  fontSize: '18px'  
}">
```

```
}">
```

Message

```
</p>
```

ngModel – Two-Way Data Binding

Requires FormsModule

```
<input [(ngModel)]="username" placeholder="Enter name" />
```

```
<p>Hello {{ username }}</p>
```

Behind the scenes

- Combines property binding + event binding

```
<input [value]="username" (input)="username = $event.target.value" />
```

3. Component Directives

Every Angular **component is a directive** with a template.

```
<app-header></app-header>
```

```
<app-footer></app-footer>
```

Component directives:

- Have templates
 - Can contain other directives
 - Are reusable UI blocks
-

Angular 20+ Considerations

1. Standalone Components (Default)

No need for NgModule to use directives.

```
@Component({
```

```
  standalone: true,
```

```
selector: 'app-demo',  
imports: [NgIf, NgFor],  
templateUrl: './demo.component.html'  
})  
  
export class DemoComponent {}
```

2. Signals Work Seamlessly with Directives

```
count = signal(0);  
  
<p *ngIf="count() > 0">  
  Count is {{ count() }}  
</p>
```

3. Desugared Syntax (Behind the *)

```
<p *ngIf="isVisible">Hello</p>  
  
Internally becomes:  
  
<ng-template [ngIf]="isVisible">  
  <p>Hello</p>  
</ng-template>
```

Understanding this helps in **advanced debugging**.

Summary Table

Category	Directives
Structural	ngIf, ngFor, ngSwitch
Attribute	ngClass, ngStyle, ngModel
Component	Custom Components
